

Grade 12 IT — Cheat Sheet

A quick "at a glance" reference. For full detail, see the **Study Guide**.

Theory in one line each

- **Data vs information** — data = raw facts; information = processed, useful data.
- **RAM** — fast, temporary, **volatile** (lost on power off). **Storage** (SSD/HDD) — permanent, **non-volatile**.
- **CPU** — the processor; speed in GHz; has cores + cache.
- **System software** (OS, utilities, drivers) runs the computer; **application software** is what you use.
- **Compiler** — translates whole program at once; **interpreter** — line by line.
- **LAN** = local (one site); **WAN** = wide (e.g. internet); **PAN** = personal (Bluetooth).
- **Router** joins networks; **switch** joins devices in one network.
- **Protocol** = communication rules: **HTTP/S** (web), **FTP** (files), **SMTP/POP3/IMAP** (email), **TCP/IP** (core internet).
- **Cloud** = store/run on the internet; **IoT** = everyday devices online.
- **Primary key** = uniquely identifies a record; **foreign key** = links to another table's primary key.
- **Validation** = data is reasonable; **verification** = data entered correctly.
- **Malware** = viruses, worms, trojans, ransomware, spyware. **Phishing** = fake messages stealing info.
- **Safeguards** — antivirus, firewall, strong passwords, 2FA, encryption, backups, updates.
- **POPIA** — SA law protecting personal information. **Copyright/piracy** — don't copy work illegally.
- **SDLC** — Analysis → Design → Coding → Testing → Implementation → Maintenance.
- **Algorithm** = step-by-step solution; **pseudocode** = plain-English steps.

Code syntax — Java vs Delphi

Concept	Java	Delphi
Assign value	<code>x = 5;</code>	<code>x := 5;</code>
Whole number	<code>int</code>	<code>Integer</code>
Decimal	<code>double</code>	<code>Real</code>
Text	<code>String</code> (double quotes)	<code>String</code> (single quotes)
True/false	<code>boolean</code>	<code>Boolean</code>
Equal / not equal	<code>== / !=</code>	<code>= / <></code>
And / or / not	<code>&& / / !</code>	<code>and / or / not</code>
Whole divide / remainder	<code>/ / %</code>	<code>div / mod</code>
Output	<code>System.out.println(x);</code>	<code>ShowMessage(x); / WriteLn(x);</code>
Text → number	<code>Integer.parseInt(s)</code>	<code>StrToInt(s)</code>
Number → text	<code>String.valueOf(n)</code>	<code>IntToStr(n)</code>
Length of text	<code>s.length()</code>	<code>Length(s)</code>

Decision (if)

```
if (age >= 18) { ... } else { ... }
```

```
if age >= 18 then ... else ...;
```

Loop (fixed number of times)

```
for (int i = 1; i <= 5; i++) { ... }
```

```
for i := 1 to 5 do begin ... end;
```

Loop (while a condition holds)

```
while (n <= 5) { ... }
```

```
while n <= 5 do begin ... end;
```

Method / function

```
public int add(int a, int b) { return a + b; }
```

```
function Add(a, b: Integer): Integer;  
begin Result := a + b; end;
```

Class (OOP) — the key ideas

- **Class** = blueprint, **object** = instance, **fields** = data, **methods** = actions.
- **Encapsulation**: fields `private` + public **get/set**.
- **Constructor** builds the object. **Inheritance** = reuse a parent class. **Polymorphism** = same method name, different behaviour.

```
public class Student {  
    private String name;  
    public Student(String n) { name = n; }  
    public String getName() { return name; }  
}
```

```
type TStudent = class  
    private FName: String;  
    public  
        constructor Create(AName: String);  
        function GetName: String;  
end;
```

Error handling (defensive programming)

```
try { age = Integer.parseInt(s); }  
catch (NumberFormatException e) { /* show message */ }
```

```
try age := StrToInt(s);  
except on E: Exception do { show message } end;
```

SQL essentials

```
SELECT * FROM Students WHERE Age > 14 ORDER BY Name;  
INSERT INTO Students (Name, Age) VALUES ('Sam', 13);  
UPDATE Students SET Age = 15 WHERE Name = 'Sam';  
DELETE FROM Students WHERE Name = 'Sam';
```

Exam quick tips

- Match detail to the **marks** (3 marks = 3 points).
- *List/name* = short; *explain/describe* = detail; *discuss* = both sides.
- Trace code with a **table of variable values**.
- Never leave a blank — always attempt.